INSTITUT TEKNOLOGI DEL
MATERI PRAKTIKUM
4231101– Dasar Pemrograman
SEMESTER I TAHUN AJAR 2026/2027

Session Date	:	Sesi 2 dan 3 Tahun 2026
Semester	:	1.
Courses	:	Dasar Pemrograman
Week/Session	:	6/1.
Key Topics	:	Looping/Iteration Statement
Activity	:	Praktikum
Duration	:	110 menit
Delivery	:	e-course
Deadline of delivery	:	**Akhir Sesi 3**
Place of delivery	:	e course
Goal	:	Mahasiswa memahami tentang materi loop dan nestedloop pada Bahasa C.

A. Nested Loop

Nested loop adalah loop di dalam loop. Untuk setiap satu iterasi loop luar, loop dalam dijalankan sampai selesai. Total iterasi body terdalam adalah iterasi luar dikali iterasi dalam.

Praktik Nested Loop: Pola Segitiga dan Piramida

Tujuan: memahami hubungan variabel loop luar dengan jumlah iterasi loop dalam.

Kode Program

Simpan sebagai NIM_nestedLoopSegitiga.c

```c
#include <stdio.h>

int main() {
    int i, j, n = 4;

    printf("Segitiga:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            printf("*");
        printf("\n");
    }

    printf("\nPiramida:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Segitiga:
*
**
***
****

Piramida:
    *
   ***
  *****
*******
```

Penjelasan Kode

- Pada segitiga, loop luar `i` mewakili baris dan loop dalam `j` berjalan dari 1 sampai `i`, sehingga baris ke-i berisi i bintang.
- `printf("\n")` berada di luar loop dalam agar pindah baris hanya setelah satu baris selesai.
- Pada piramida, ada dua loop dalam berurutan: pertama mencetak `n - i` spasi, kemudian mencetak `2 * i - 1` bintang.

Rumus piramida (n = 4)

Baris i	Spasi (n - i)	Bintang (2i - 1)
1	3	1
2	2	3
3	1	5
4	0	7

Penjelasan Hasil

- Segitiga tercetak 1, 2, 3, 4 bintang per baris. Totalnya 1 + 2 + 3 + 4 = 10 bintang, dan `printf("*")` dipanggil 10 kali.
- Piramida memiliki pola spasi 3, 2, 1, 0 dan bintang 1, 3, 5, 7, sehingga bentuknya simetris dan rata tengah.

Praktik Nested Loop: Tabel Perkalian 1-5

Tujuan: membaca nested loop sebagai baris dan kolom.

Kode Program

Simpan sebagai NIM_nestedLoopPerkalian.c

```c
#include <stdio.h>

int main() {
    int i, j;

    for (i = 1; i <= 5; i++) {          /* loop luar: baris */
        for (j = 1; j <= 5; j++) {      /* loop dalam: kolom */
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

Penjelasan Kode

- Loop luar `i` mengatur baris dan loop dalam `j` mengatur kolom. Sel pada baris i dan kolom j berisi `i * j`.
- `%4d` memberi lebar 4 karakter untuk setiap angka agar kolom sejajar.
- `printf("\n")` dijalankan setelah loop dalam selesai, sekali per baris.

Penjelasan Hasil

- Hasil berupa matriks 5 x 5. Body terdalam dijalankan 5 x 5 = 25 kali.
- Baris pertama 1..5 dan baris terakhir 5, 10, ..., 25.
- Tabel simetris terhadap diagonal karena i * j = j * i.

B. Pernyataan break

`break` menghentikan struktur kontrol paling dalam yang sedang berjalan. Eksekusi lanjut ke pernyataan setelah struktur yang ditinggalkan.

Praktik Break: break pada Loop Tunggal dan Nested Loop

Tujuan: membedakan efek `break` pada loop tunggal dan pada loop dalam.

Kode Program

Simpan sebagai NIM_breakLoop.c

```c
#include <stdio.h>

int main() {
    int i, j;

    printf("Contoh 1 (break pada loop tunggal):\n");
    for (i = 0; i < 5; i++) {
        if (i == 2) {
            break;
        }
        printf("%d\n", i);
    }

    printf("Contoh 2 (break pada nested loop):\n");
    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) {
            if (j == 2)
                break;
            printf("%d\n", i + j);
        }
    }
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Contoh 1 (break pada loop tunggal):
0
1
Contoh 2 (break pada nested loop):
0
1
1
2
2
3
3
4
4
5
```

Penjelasan Kode

- Contoh 1: ketika `i == 2`, `break` langsung keluar dari loop `for`. Nilai 2, 3, 4 tidak pernah diproses.
- Contoh 2: `break` berada di loop dalam (`j`), sehingga hanya loop `j` yang berhenti. Loop luar `i` tetap berjalan sampai 4.
- Pada contoh 2, `j` hanya mencapai 0 dan 1 karena berhenti saat `j == 2`.

Penelusuran Contoh 2

i	j yang dicetak	Nilai i + j
0	0, 1	0, 1
1	0, 1	1, 2
2	0, 1	2, 3
3	0, 1	3, 4

i	j yang dicetak	Nilai i + j
4	0, 1	4, 5

Penjelasan Hasil

- Contoh 1 mencetak 0 dan 1 saja, karena loop berhenti saat `i` bernilai 2.
- Contoh 2 mencetak 10 angka (5 nilai `i` x 2 nilai `j`): 0 1, 1 2, 2 3, 3 4, 4 5.
- Bandingkan kedua contoh: `break` hanya memengaruhi loop terdekat yang membungkusnya.

Praktik Break: break untuk Mencari Bilangan Prima

Tujuan: memakai `break` agar pencarian berhenti begitu pembagi ditemukan.

Kode Program

Simpan sebagai NIM_breakPrima.c

```c
#include <stdio.h>

int main() {
    int n, i, prima = 1;

    printf("Masukkan bilangan (>= 2): ");
    scanf("%d", &n);

    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            prima = 0;
            printf("%d habis dibagi %d, berhenti memeriksa.\n", n, i);
            break;
        }
    }
    if (prima)
        printf("%d adalah bilangan prima.\n", n);
    else
        printf("%d bukan bilangan prima.\n", n);
    return 0;
}
```

Hasil Eksekusi (dua kali percobaan)

Teks kuning tebal adalah masukan yang diketik pengguna.

Percobaan 1: n = 29

```
Masukkan bilangan (>= 2): 29
29 adalah bilangan prima.
```

Percobaan 2: n = 15

```
Masukkan bilangan (>= 2): 15
15 habis dibagi 3, berhenti memeriksa.
15 bukan bilangan prima.
```

Penjelasan Kode

- `prima = 1` adalah variabel penanda (flag). Anggap n prima sampai ditemukan pembagi.
- Kondisi `i * i <= n` cukup, karena bila n punya pembagi lebih besar dari akar n, pasti ada pasangannya yang lebih kecil. Ini mengurangi jumlah iterasi.

- Saat `n % i == 0`, flag diubah menjadi 0, pesan dicetak, dan `break` menghentikan loop karena memeriksa pembagi lain tidak diperlukan lagi.

Penjelasan Hasil

- n = 29: `i` diperiksa 2, 3, 4, 5 (karena 6 x 6 = 36 > 29). Tidak ada yang habis membagi, flag tetap 1, sehingga 29 prima.
- n = 15: `i = 2` tidak membagi (15 % 2 = 1). Pada `i = 3`, 15 % 3 = 0, flag menjadi 0, `break` dijalankan, dan 15 bukan prima.
- Pada kasus 15, loop berhenti di iterasi ke-2 dan tidak menunggu sampai `i * i > 15`. Inilah keuntungan `break`.

C. Pernyataan continue

`continue` melewati sisa body pada iterasi saat ini dan langsung memulai iterasi berikutnya. Pada `for`, bagian update tetap dijalankan sebelum kondisi dicek lagi.

Praktik Continue: continue pada Loop Tunggal dan Nested Loop

Tujuan: memahami bahwa `continue` hanya melewati satu iterasi, bukan menghentikan loop.

Kode Program

Simpan sebagai NIM_continueLoop.c

```c
#include <stdio.h>

int main(void) {
    int i, j;

    printf("Contoh 1 (continue pada loop tunggal):\n");
    for (i = 0; i < 5; i++) {
        if (i == 2)
            continue;
        printf("%d\n", i);
    }

    printf("Contoh 2 (continue pada nested loop):\n");
    for (i = 0; i < 5; i++) {
        for (j = 1; j < 5; j++) {
            if (j == 2)
                continue;
            printf("%d\n", i + j);
        }
    }
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Contoh 1 (continue pada loop tunggal):
0
1
3
4
Contoh 2 (continue pada nested loop):
1
3
4
2
4
5
3
5
6
4
6
7
5
7
8
```

Penjelasan Kode

- Contoh 1: saat `i == 2`, `continue` melewati `printf`, lalu `i++` berjalan dan loop lanjut dengan `i = 3`.
- Contoh 2: loop dalam dimulai dari `j = 1`. Nilai `j == 2` dilewati, sehingga yang diproses hanya `j = 1, 3, 4`.

Penelusuran Contoh 2

i	j yang diproses (2 dilewati)	Nilai i + j
0	1, 3, 4	1, 3, 4
1	1, 3, 4	2, 4, 5
2	1, 3, 4	3, 5, 6
3	1, 3, 4	4, 6, 7
4	1, 3, 4	5, 7, 8

Penjelasan Hasil

- Contoh 1 menghasilkan 0, 1, 3, 4. Angka 2 hilang tetapi loop tetap berjalan sampai 4.
- Contoh 2 menghasilkan 15 angka (5 nilai `i` x 3 nilai `j`), sesuai tabel penelusuran.
- Perbedaan inti: `break` mengakhiri loop, sedangkan `continue` hanya melewati satu iterasi.

Praktik Continue: continue untuk Menjumlah Bilangan Ganjil

Tujuan: memakai `continue` sebagai filter data.

Kode Program

Simpan sebagai NIM_continueGanjil.c

```c
#include <stdio.h>

int main() {
    int i, jumlah = 0;

    for (i = 1; i <= 10; i++) {
        if (i % 2 == 0)
            continue;               /* lewati bilangan genap */
        jumlah += i;
        printf("i = %2d (ganjil) -> jumlah = %d\n", i, jumlah);
    }
    printf("Jumlah bilangan ganjil 1..10 = %d\n", jumlah);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
i =  1 (ganjil) -> jumlah = 1
i =  3 (ganjil) -> jumlah = 4
i =  5 (ganjil) -> jumlah = 9
i =  7 (ganjil) -> jumlah = 16
i =  9 (ganjil) -> jumlah = 25
Jumlah bilangan ganjil 1..10 = 25
```

Penjelasan Kode

- `if (i % 2 == 0) continue;` melewati bilangan genap sehingga tidak ikut dijumlahkan dan tidak dicetak.
- Hanya bilangan ganjil yang mencapai `jumlah += i`.
- Total dicetak di luar loop setelah semua iterasi selesai.

Penjelasan Hasil

- Hanya baris untuk i = 1, 3, 5, 7, 9 yang muncul. Bilangan genap tidak tampil.
- Jumlah bertahap 1, 4, 9, 16, 25. Totalnya 25, yaitu 5 x 5, sesuai sifat jumlah n bilangan ganjil pertama = n kuadrat.

Catatan: Hati-hati memakai `continue` pada `while`. Jika `continue` dieksekusi sebelum pernyataan update variabel, update dilewati dan loop dapat menjadi infinite loop.

D. Kesalahan Umum pada Loop

Praktik Kesalahan: Off-by-One dan Titik Koma Ekstra

Tujuan: mengenali dua kesalahan loop yang paling sering terjadi.

Kode Program

Simpan sebagai m1.c

```c
#include <stdio.h>

int main() {
    int i;

    printf("A. Off-by-one: i < 5\n");
    for (i = 1; i < 5; i++)
        printf("%d ", i);

    printf("\nB. Off-by-one diperbaiki: i <= 5\n");
    for (i = 1; i <= 5; i++)
        printf("%d ", i);

    printf("\nC. Titik koma ekstra setelah for:\n");
    for (i = 0; i < 3; i++);
        printf("Halo, i = %d\n", i);
    return 0;
}
```

Hasil Eksekusi

```
A. Off-by-one: i < 5
1 2 3 4
B. Off-by-one diperbaiki: i <= 5
1 2 3 4 5
C. Titik koma ekstra setelah for:
Halo, i = 3
```

Penjelasan Kode

- Bagian A: `i < 5` berhenti di 4, sehingga angka 5 hilang. Ini disebut kesalahan off-by-one. Bagian B memakai `i <= 5` dan hasilnya benar.
- Bagian C: tanda `;` tepat setelah `for (...)` menjadikan body loop kosong. Loop tetap berjalan 3 kali tetapi tidak melakukan apa-apa.
- `printf` yang menjorok (indentasi) bukan bagian loop. Ia hanya dijalankan sekali setelah loop selesai, ketika `i` sudah bernilai 3.

Penjelasan Hasil

- A mencetak 1 2 3 4, B mencetak 1 2 3 4 5. Satu simbol (`<` menjadi `<=`) mengubah jumlah iterasi.
- C hanya mencetak satu baris "Halo, i = 3", padahal programmer mungkin mengharapkan tiga baris.
- Kompiler tidak menolak titik koma ekstra karena sintaksnya sah. Kesalahan ini hanya terlihat dari hasil yang tidak sesuai harapan.

Titik Koma Ekstra pada while (jangan dijalankan)

```c
int num = 0, minimum = 5;
while (num < minimum);       /* ; di sini = body kosong */
{
    scanf("%d", &num);
    printf("Coba lagi.\n");
}
```

Pada `while`, titik koma ekstra menyebabkan infinite loop: body kosong tidak pernah mengubah `num`, sehingga kondisi `num < minimum` selalu true. Blok `{ ... }` di bawahnya tidak pernah tercapai. Program akan terus berjalan sampai dihentikan paksa (Ctrl+C).

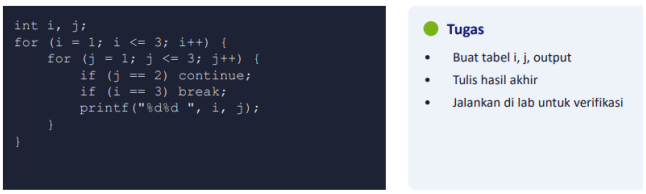
Daftar Periksa Kesalahan Loop

Gejala	Penyebab Umum	Perbaikan
Program tidak berhenti	Variabel kontrol tidak diupdate, atau `;` setelah `while`	Pastikan ada update dan hapus `;` ekstra
Jumlah iterasi kurang atau lebih satu	`<` vs `<=`, awal 0 vs 1	Telusuri nilai awal dan akhir dengan tabel
Hasil akumulasi aneh	Akumulator belum diinisialisasi	Beri nilai awal (0 untuk jumlah, 1 untuk perkalian)
`do-while` error kompilasi	Lupa `;` setelah `while (...)`	Tambahkan `;`
Rata-rata tidak berdesimal	Pembagian bilangan bulat	Gunakan cast `(double)`

Latihan Mandiri Pertemuan 2

1. Cetak pola segitiga terbalik (baris pertama 5 bintang, berkurang satu tiap baris).
2. Cetak seluruh bilangan prima dari 2 sampai 50 dengan nested loop dan `break`.
3. Cetak angka 1 sampai 20 tetapi lewati kelipatan 3 memakai `continue`.
4. Tebak hasil kode pada latihan tracing berikut:

   Latihan Tracing: Prediksi Output

   Lalu jalankan: `for i=1..3`, `for j=1..3`, `if (j==2) continue; if (i==3) break; printf("%d%d ", i, j);` (jawaban akan Anda verifikasi sendiri di lab).
5. Buatkan semua code tugas 1-4 ke dalam flowgorithm.